

OpenCatESP32 – Quadruped Robot Firmware

Table of Contents

- [Overview](#)
 - [Features](#)
 - [Hardware Requirements](#)
 - [Wiring Diagram](#)
 - [Servo Mapping](#)
 - [Software Setup](#)
 - [Compilation & Upload](#)
 - [Getting Started](#)
 - [Bluetooth Commands](#)
 - [Serial Commands](#)
 - [Calibration Guide](#)
 - [Architecture](#)
 - [Module Reference](#)
 - [Skill System](#)
 - [Gait Engine](#)
 - [Configuration Reference](#)
 - [Troubleshooting](#)
 - [Adding New Skills](#)
 - [Future Expansion](#)
 - [Project Files](#)
-

Overview

OpenCatESP32 is a complete quadruped robot firmware designed to run on an **ESP32 DevKit V1** controlling **8 servos** via a **PCA9685** PWM driver, with wireless control through **Bluetooth Classic**.

The firmware is inspired by Peto OpenCat but built entirely from scratch for this specific hardware combination. It features a layered architecture with clean separation of concerns, making it easy to extend and maintain.

What Makes This Different

Feature	OpenCatESP32	Typical Hobby Projects
Architecture	Layered, modular, class-based	Single-file, procedural
Motion	Interpolated keyframes + procedural gaits	Raw <code>servo.write()</code> calls
Timing	FreeRTOS tasks, <code>millis()</code> -based delta time	<code>delay()</code> blocking
Safety	Servo limits, timeout, emergency stop	None
Calibration	NVS-persistent, live-adjustable offsets	Hardcoded
Code Quality	Doxygen documented, namespaced config	Magic numbers

Features

Motion System

- **13 pre-programmed skills** — stand, sit, lay, sleep, walk, backward, turn left/right, trot, crawl, wave, shake, stretch
- **6 interpolation curves** — linear, cosine, cubic, ease-in, ease-out, ease-in-out
- **Smooth transitions** — blended pose changes between any two states
- **Procedural gait engine** — phase-based walking with configurable stride, step height, and body sway
- **5 gait types** — walk (crawl), trot, turn left, turn right, backward

Control

- **Bluetooth Classic** — single-character commands from Android app
- **Serial control** — same commands via USB for testing without Bluetooth
- **Non-blocking** — all I/O is polled, never blocking

Safety & Reliability

- **Per-servo angle limits** — prevents mechanical over-travel
- **Calibration offset clamping** — $\pm 30^\circ$ maximum offset
- **Servo timeout** — auto-disable after 30 seconds of inactivity
- **Emergency stop** — immediate halt on `S` command
- **Brownout protection hooks** — ready for voltage monitoring
- **I2C scanner** — built-in diagnostic for wiring problems

Quality

- **FreeRTOS dual-core** – Motion on Core 1(50 Hz), Bluetooth on Core 0
 - **Persistent calibration** – NVS storage survives power cycles
 - **Zero `delay()` calls** in motion code – all timing is delta-time based
 - **Fully documented** – Doxygen comments on every class and method
 - **Configurable everything** – single `Config.h` with namespaced parameters
-

Hardware Requirements

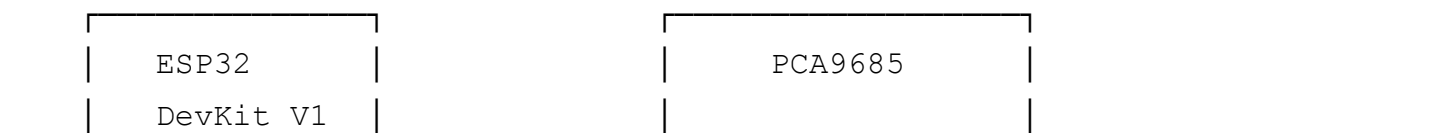
Component	Model	Quantity	Notes
Microcontroller	ESP32 DevKit V1(ESP-WR00M-32)	1	30-pin or 38-pin version
Servo Driver	PCA9685 16-Channel PWM Driver	1	I2C address 0x40 (default)
Servos	SG90 or MG90S Micro Servos	8	180° range, 4.8–6V
Power Supply	5V DC, ≥3A	1	For servo power only
Wires	Dupont jumper wires	~12	Male-to-female or as needed
USB Cable	Micro-USB	1	For programming and serial debug

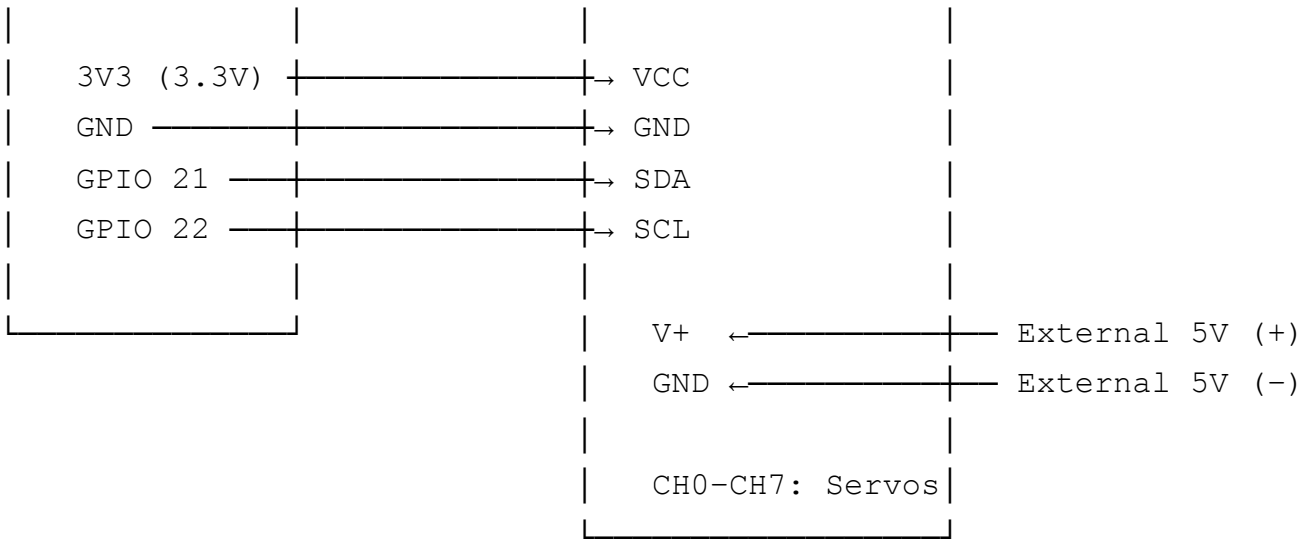
Power Notes

- CRITICAL:** Never power servos from the ESP32's 5V pin. 8 servos can draw 2–4A under load, which will cause brownouts and damage the ESP32. Always use an external 5V power supply connected to the PCA9685's V+ terminal.
- CRITICAL:** The ESP32, PCA9685, and external servo power supply **must share a common ground**. Without this, I2C communication will fail and servos will behave erratically.
-

Wiring Diagram

ESP32 to PCA9685 (4 wires)





Connection Table

ESP32 Pin	PCA9685 Pin	Purpose
3V3	VCC	Logic power for PCA9685 chip
GND	GND	Common ground
GPIO 21	SDA	I2C data
GPIO 22	SCL	I2C clock
—	V+	External 5V servo power (NOT from ESP32)

PCA9685 to Servos (8 channels)

Each servo connects to a 3-pin header on the PCA9685 board:

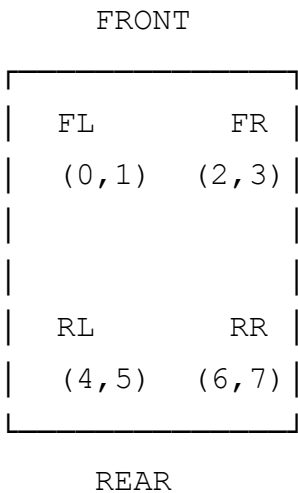
- **Yellow/Orange** wire → PWM signal
- **Red** wire → V+ (power)
- **Brown/Black** wire → GND

PCA9685 Channel	Servo	Robot Position
CH0	Servo 1	Front Left Hip
CH1	Servo 2	Front Left Knee
CH2	Servo 3	Front Right Hip
CH3	Servo 4	Front Right Knee
CH4	Servo 5	Rear Left Hip
CH5	Servo 6	Rear Left Knee

PCA9685 Channel	Servo	Robot Position
CH6	Servo 7	Rear Right Hip
CH7	Servo 8	Rear Right Knee

Servo Mapping

The robot has 4 legs, each with 2 joints (hip and knee), totaling 8 servos.



FL = Front Left FR = Front Right
 RL = Rear Left RR = Rear Right

Even channels (0, 2, 4, 6) = Hip joints
 Odd channels (1, 3, 5, 7) = Knee joints

Servo Direction Convention

- **Hip at 90°** = Leg pointing straight down (neutral)
- **Hip > 90°** = Leg swings forward
- **Hip < 90°** = Leg swings backward
- **Knee at 45°** = Leg extended (standing)
- **Knee at 130°** = Leg fully folded (laying)

Right-side servos (CH2, CH3, CH6, CH7) are **inverted** by default in `Config.h` to account for mirror mounting. If your servos move the wrong direction, toggle the `INVERTED[]` flags.

Software Setup

1. Install Arduino IDE

Download and install [Arduino IDE 2.x](#) (recommended) or Arduino IDE 1.8.x.

2. Add ESP32 Board Support

1. Open Arduino IDE
2. Go to **File → Preferences**
3. In **"Additional Board Manager URLs"**, add:

```
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json
```
4. Go to **Tools → Board → Board Manager**
5. Search for **"ESP32"** and install **"ESP32 by Espressif Systems"**

3. Install Required Libraries

Go to **Sketch → Include Library → Manage Libraries** and install:

Library	Author	Version	Purpose
Adafruit PWM Servo Driver Library	Adafruit	Latest	PCA9685 I2C communication

The following libraries are **built-in** (no installation needed):

- `Wire` – I2C communication
- `BluetoothSerial` – ESP32 Bluetooth Classic
- `Preferences` – Non-volatile storage (NVS)

4. Board Configuration

Go to **Tools** and set:

Setting	Value
Board	ESP32 Dev Module
Upload Speed	921600
CPU Frequency	240MHz (WiFi/BT)
Flash Frequency	80MHz
Flash Mode	QIO
Flash Size	4MB (32Mb)

Step 1: Test Servos

Type **t** in Serial Monitor. All 8 servos should snap to 90° (center position).

Step 2: Calibrate

Type **c** to enter calibration mode. Adjust each servo so the legs are symmetric and the robot stands level. See the [Calibration Guide](#).

Step 3: Test via Serial

Type these characters in Serial Monitor to test each pose:

- **A** — Stand
- **M** — Sit
- **D** — Lay
- **F** — Walk forward

Step 4: Connect Android App

1. Pair your phone to **"OpenCat ESP32"** via Bluetooth settings
2. Open your Android control app
3. Send commands — the robot should respond in real-time

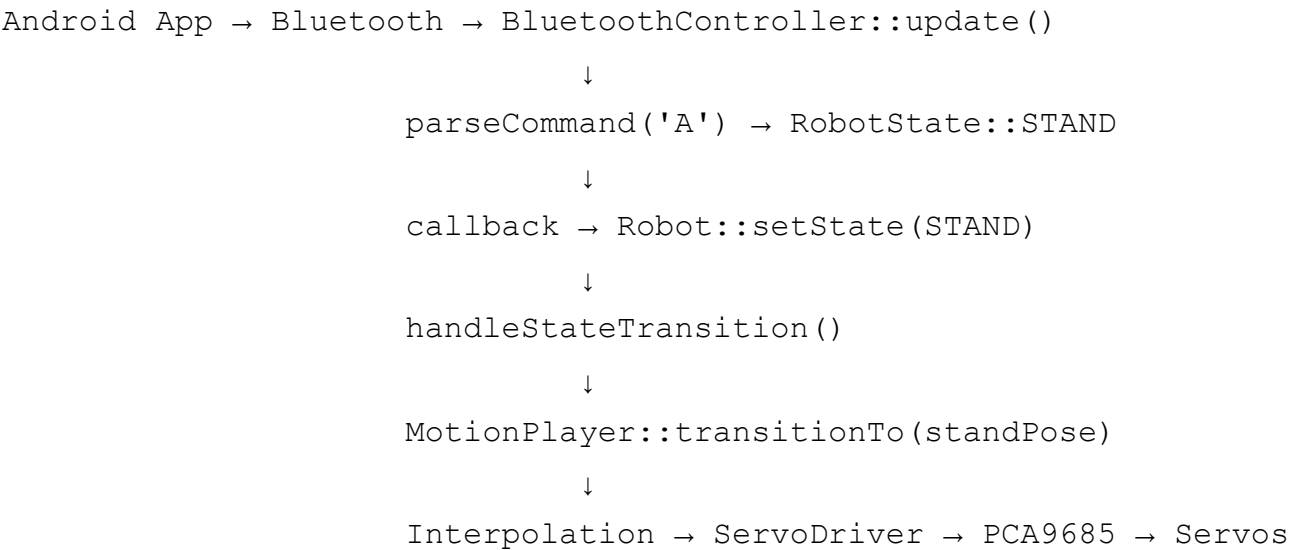
Bluetooth Commands

These single-character commands match the existing Android application:

Command	Action	Motion Type	Description
A	Stand	Static pose	All legs straight, body raised
M	Sit	Static pose	Rear legs fold, front stays up
D	Lay	Static pose	All legs fold, body flat
C	Sleep	Static pose	Legs tucked tight, compact
F	Walk Forward	Gait engine	Continuous forward crawl gait
B	Walk Backward	Gait engine	Continuous backward gait
L	Turn Left	Gait engine	Differential turn (right steps more)

Command	Action	Motion Type	Description
R	Turn Right	Gait engine	Differential turn (left steps more)
S	Stop	Idle	Stop all motion, hold position

How Commands Are Processed



Serial Commands

Type these in the Arduino Serial Monitor (115200 baud):

Key	Action	Description
c	Calibration mode	Enter interactive servo calibration
i	System info	Print state, connections, memory usage
t	Test servos	Center all servos at 90°
p	Print offsets	Show current calibration offsets
k	List skills	Show all registered skills
w	I2C scan	Scan I2C bus for connected devices
A/M/D/C/F/B/L/R/S	Robot commands	Same as Bluetooth (for testing)

Calibration Guide

Calibration adjusts per-servo angle offsets so the robot's legs are symmetric and properly aligned. Offsets are stored in ESP32's non-volatile storage (NVS)

and survive power cycles.

Why Calibrate?

SG90/MG90S servos have manufacturing tolerances. When you command 90°, each servo might actually be at 87° or 93°. Calibration offsets compensate for this.

Calibration Procedure

1. **Enter calibration mode:** Type `c` in Serial Monitor
2. **All servos center at 90°** + their current offsets
3. **Select a servo:** Type `0` through `7` to select a channel
4. **Adjust offset:** Press `+` or `-` to adjust by $\pm 1^\circ$
5. **Observe the servo:** It moves immediately so you can see the effect
6. **Repeat** for all 8 servos
7. **Save:** Press `s` to write offsets to NVS
8. **Exit:** Press `x` to return to normal operation

Calibration Commands

Key	Action
<code>0 - 7</code>	Select servo channel
<code>+</code> or <code>=</code>	Increase offset by $+1^\circ$
<code>-</code> or <code>_</code>	Decrease offset by -1°
<code>s</code>	Save all offsets to NVS
<code>r</code>	Reset all offsets to 0°
<code>p</code>	Print current offset table
<code>h</code>	Show help menu
<code>x</code>	Exit calibration mode

Calibration Tips

- Start with the robot on its back so legs move freely
- Each hip servo should be at the same angle — legs should be symmetric left-to-right
- Each knee servo should match its pair — front knees the same, rear knees the same
- After calibrating, flip the robot over and test `A` (stand) — it should be level

- Maximum offset is $\pm 30^\circ$ – if you need more, your servo horn is mounted wrong

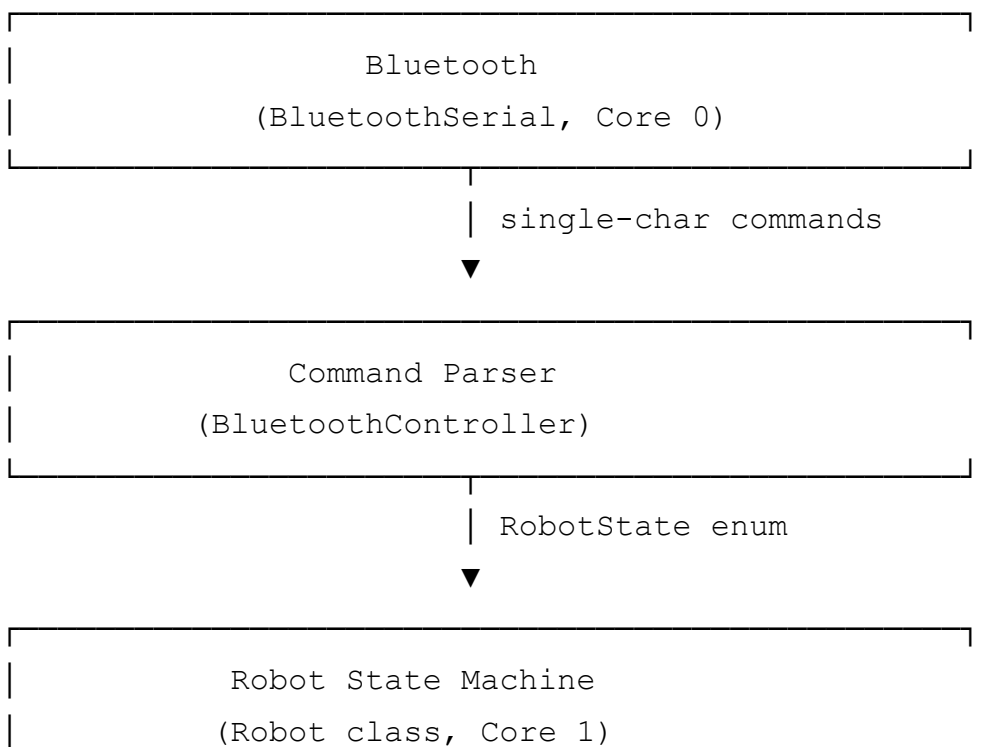
Offset Table Example

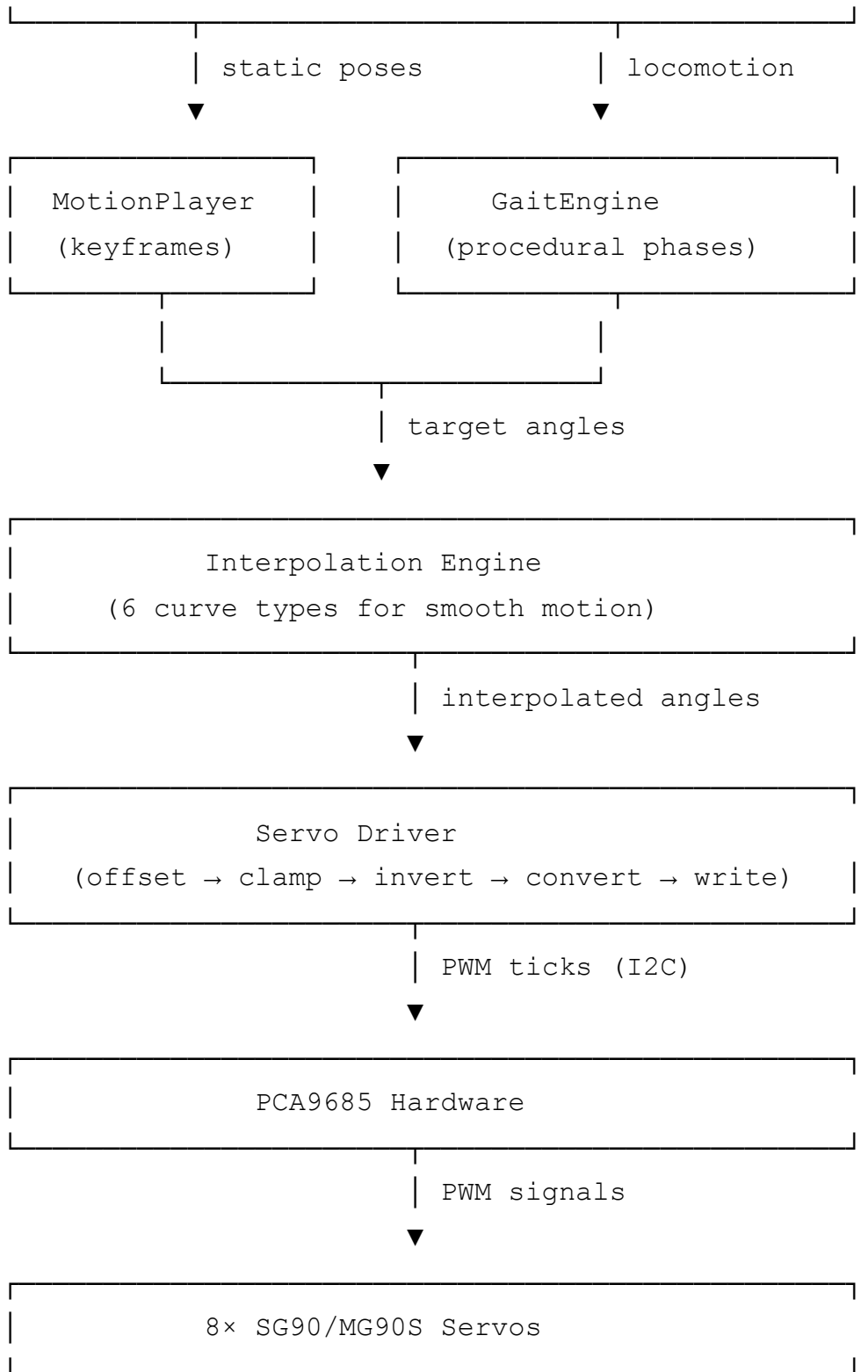
CH	Offset	Servo
0	+2.0°	FL Hip
1	-1.0°	FL Knee
2	+3.0°	FR Hip
3	0.0°	FR Knee
4	-2.0°	RL Hip
5	+1.0°	RL Knee
6	0.0°	RR Hip
7	-1.0°	RR Knee

Architecture

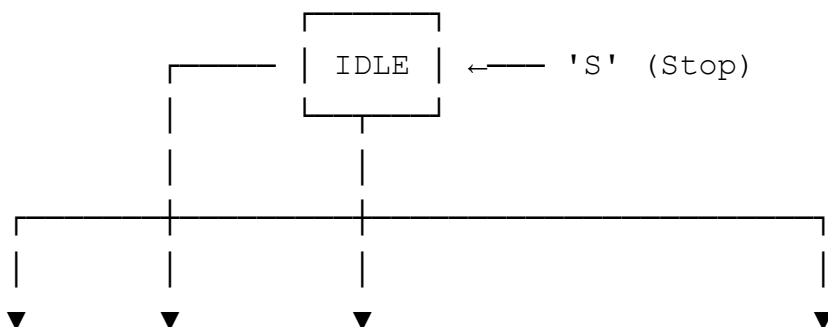
Layered Design

The firmware follows a strict layered architecture. Each layer only communicates with the layer directly below it. Bluetooth never touches servos directly – it only changes robot state.





State Machine



STAND	SIT	LAY	SLEEP
'A'	'M'	'D'	'C'

Static Poses
(MotionPlayer)

WALK_FWD
WALK_BACK
TURN_LEFT
TURN_RIGHT
TROT/CRAWL

Locomotion
(GaitEngine)

FreeRTOS Task Architecture

Task	Core	Frequency	Priority	Stack	Responsibility
Motion Task	Core 1	50 Hz	2 (high)	4 KB	State machine, servo output
Bluetooth Task	Core 0	100 Hz	1 (low)	4 KB	BT polling, command parsing
Arduino loop()	Core 1	~20 Hz	0 (idle)	Default	Serial debug commands

Module Reference

Constants.h

Defines all enumerations (`ServoID` , `LegID` , `JointType` , `RobotState` , `InterpolationType` , `GaitType`) and helper functions for servo↔leg mapping.

Config.h

Master configuration file. All tunable parameters organized into namespaces:

`I2C` , `PCA9685` , `Servo` , `Leg` , `Gait` , `Bluetooth` , `Motion` , `Safety` , `Tasks` , `Calibration` , `Serial_` .

Utils.h / Utils.cpp

Float-safe math (`clampf` , `mapf` , `lerpf`), unit conversions (degrees↔microseconds↔ticks, degrees↔radians), timing helpers, debug macros.

ServoDriver.h / ServoDriver.cpp

PCA9685 hardware abstraction. Angle processing pipeline:

`offset` → `safety clamp` → `inversion` → `hardware clamp` → `µs` → `ticks` → `PCA9685`.

Per-servo state tracking, enable/disable, calibration offset support.

Calibration.h / Calibration.cpp

NVS-persistent servo offsets. Interactive serial calibration menu with live servo feedback. Save/load/reset functionality.

Interpolation.h / Interpolation.cpp

Six interpolation curve functions (all take normalized $t \in [0, 1]$):
linear, cosine, cubic (Hermite), ease-in (quadratic), ease-out (quadratic),
ease-in-out (cubic).

MotionEngine.h / MotionEngine.cpp

Keyframe-based motion playback. Data structures: `Pose` (8 angles),
`Keyframe` (pose + duration + curve), `Motion` (keyframe array + loop flag).
Plays motions with interpolated transitions between keyframes.

MotionPlayer.h / MotionPlayer.cpp

High-level orchestrator. Smooth transitions between any two poses.
Motion queuing. Sits above MotionEngine and handles blending.

Skills.h / Skills.cpp

Skill registry — maps string names to `Motion` definitions. Registers all
13 built-in skills at startup. Extensible via `registerSkill()`.

Gaits.h / Gaits.cpp

Procedural gait engine. Phase-based leg coordination with configurable
stride, step height, cycle duration, and body sway. Swing/stance phase
decomposition with smooth ramp-up/ramp-down.

Bluetooth.h / Bluetooth.cpp

Wraps `BluetoothSerial` . Non-blocking character polling, single-char command parsing, callback dispatch to Robot state machine. Runs on Core 0.

Robot.h / Robot.cpp

Central coordinator. Owns all subsystems. State machine determines whether MotionPlayer (static poses) or GaitEngine (locomotion) is active. Handles transitions, safety timeout, and emergency stop.

Skill System

Skills are pre-defined motion sequences stored as `const Motion` structs in header files under the `skills/` directory.

Built-in Skills

Skill	File	Keyframes	Loop	Description
Stand	<code>skills/stand.h</code>	1	No	Neutral standing pose
Sit	<code>skills/sit.h</code>	2	No	Rear legs fold down
Lay	<code>skills/lay.h</code>	2	No	All legs fold flat
Sleep	<code>skills/sleep.h</code>	2	No	Compact tucked position
Walk	<code>skills/walk.h</code>	8	Yes	Crawl gait (one leg at a time)
Back	<code>skills/back.h</code>	8	Yes	Reverse crawl
Left	<code>skills/left.h</code>	4	Yes	Differential left turn
Right	<code>skills/right.h</code>	4	Yes	Differential right turn
Trot	<code>skills/trot.h</code>	4	Yes	Diagonal pair gait
Crawl	<code>skills/crawl.h</code>	8	Yes	Slow, stable crawl
Wave	<code>skills/wave.h</code>	4	No	Wave front-right paw
Shake	<code>skills/shake.h</code>	6	No	Handshake gesture
Stretch	<code>skills/stretch.h</code>	4	No	Cat-like stretch

Skill Data Format

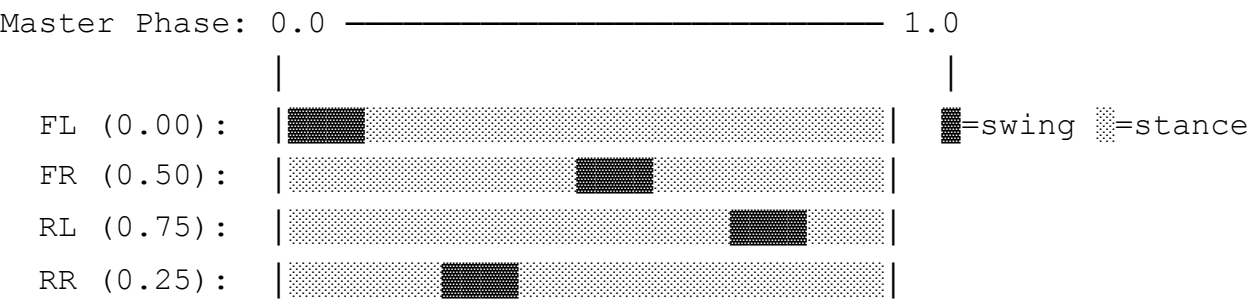
```
static const Keyframe standKeyframes[] = {
    {
        .pose = { .angles = {
            90.0f,    // CH0: FL Hip
            45.0f,    // CH1: FL Knee
            90.0f,    // CH2: FR Hip
            45.0f,    // CH3: FR Knee
            90.0f,    // CH4: RL Hip
            45.0f,    // CH5: RL Knee
            90.0f,    // CH6: RR Hip
            45.0f     // CH7: RR Knee
        }},
        .durationMs      = 500,
        .interpolation = InterpolationType::EASE_IN_OUT
    }
};
```

Gait Engine

The gait engine generates walking motions **procedurally** (not from pre-recorded keyframes). This allows runtime adjustment of speed, stride length, and turning radius.

How It Works

Each leg has a **phase offset** (0.0-1.0) that determines when it swings vs. stands relative to the master gait cycle.



Gait Patterns

Gait	Phase Offsets (FL, FR, RL, RR)	Description
Walk	0.0, 0.5, 0.75, 0.25	One leg at a time (most stable)

Gait	Phase Offsets (FL, FR, RL, RR)	Description
Trot	0.0, 0.5, 0.5, 0.0	Diagonal pairs (faster)
Turn Left	Walk pattern + right legs stride full, left stride reduced	
Turn Right	Walk pattern + left legs stride full, right stride reduced	
Backward	Walk pattern with negated stride direction	

Gait Parameters (Config.h)

Parameter	Default	Description
STRIDE_DEG	30°	Hip swing amplitude
STEP_HEIGHT_DEG	25°	Knee lift during swing
CYCLE_DURATION_MS	1200 ms	Walk cycle period
TROT_CYCLE_MS	600 ms	Trot cycle period (faster)
TURN_INNER_FACTOR	0.3	Inner leg stride reduction
BODY_SWAY_DEG	3°	Lateral weight shift
RAMP_DURATION_MS	400 ms	Speed ramp up/down time
BODY_HEIGHT_DEG	45°	Standing knee angle

Configuration Reference

All configuration lives in `Config.h`. Edit these values to tune the robot:

Servo Limits

```
Config::Servo::PULSE_MIN_US = 600;    // 0° pulse width
Config::Servo::PULSE_MAX_US = 2400;   // 180° pulse width
Config::Servo::SAFE_MIN[8]  = {10°};  // Per-servo minimum angle
Config::Servo::SAFE_MAX[8]  = {170°}; // Per-servo maximum angle
Config::Servo::INVERTED[8]  = {...};  // Direction inversion flags
```

I2C

```
Config::I2C::SDA_PIN    = 21;        // ESP32 I2C data pin
Config::I2C::SCL_PIN    = 22;        // ESP32 I2C clock pin
Config::I2C::CLOCK_HZ   = 400000;    // 400 kHz fast mode
Config::PCA9685::ADDRESS = 0x40;     // Default PCA9685 address
```

Bluetooth

```
Config::Bluetooth::DEVICE_NAME      = "OpenCat ESP32";
Config::Bluetooth::POLL_INTERVAL_MS = 10;    // 100 Hz polling
```

Safety

```
Config::Safety::SERVO_TIMEOUT_MS      = 30000; // 30s → disable servos
Config::Safety::MAX_SPEED_DEG_PER_SEC = 300;   // Rate limiter
Config::Safety::WATCHDOG_TIMEOUT_SEC   = 5;     // Watchdog timer
```

Troubleshooting

PCA9685 Not Detected

Symptom: [ServoDriver] PCA9685 NOT found at 0x40

Diagnosis: Type `w` in Serial Monitor to run I2C bus scan.

Scan Result	Cause	Fix
No devices found	Wiring problem	Check VCC (3.3V), GND, SDA (21), SCL (22)
Device at different address	Address jumpers soldered	Update <code>Config::PCA9685::ADDRESS</code>
Device at 0x40 found but init fails	Timing issue	Add <code>delay(100)</code> before <code>begin()</code>

Servos Not Moving

Symptom	Cause	Fix
---------	-------	-----

Symptom	Cause	Fix
PCA9685 detected but no movement	No external 5V on V+	Connect 5V power to V+ terminal
Servos jitter/buzz	Stall at limit	Widen <code>SAFE_MIN</code> / <code>SAFE_MAX</code> in Config.h
Servos move wrong direction	Mirror mounting	Toggle <code>INVERTED[]</code> flag in Config.h
Only some servos work	Bad connection	Check servo cables on those channels

Bluetooth Issues

Symptom	Cause	Fix
Can't find "OpenCat ESP32"	BT init failed	Check Serial Monitor for BT errors
Connects but no response	Wrong command chars	Ensure app sends uppercase A/M/D/C/F/B/L/R/S
Delayed response	Buffer overflow	Reduce command rate from app

Compilation Errors

Error	Fix
<code>BluetoothSerial.h</code> not found	Install ESP32 board support package
<code>Adafruit_PWMServoDriver.h</code> not found	Install Adafruit PWM Servo Driver library
Multiple definitions	Ensure only one <code>.ino</code> file in the folder

Adding New Skills

Step 1: Create the Skill File

Create `skills/mykill.h`:

```
#ifndef SKILL_MYSKILL_H
#define SKILL_MYSKILL_H

#include "../MotionEngine.h"
```

```

namespace Skills {

    static const Keyframe mySkillKeyframes[] = {
        // Keyframe 1: Starting pose
        {
            .pose = { .angles = {
                90.0f, 45.0f,    // FL Hip, Knee
                90.0f, 45.0f,    // FR Hip, Knee
                90.0f, 45.0f,    // RL Hip, Knee
                90.0f, 45.0f     // RR Hip, Knee
            }},
            .durationMs      = 500,
            .interpolation = InterpolationType::EASE_IN_OUT
        },
        // Add more keyframes...
    };

    static const Motion mySkillMotion = {
        .name          = "myskill",
        .keyframes      = mySkillKeyframes,
        .numKeyframes   = sizeof(mySkillKeyframes) / sizeof(mySkillKeyframe),
        .loop           = false // true for repeating motions
    };

} // namespace Skills

#endif

```

Step 2: Register in Skills.cpp

Add to the includes:

```
#include "skills/myskill.h"
```

Add to `registerBuiltins()` :

```
registerSkill("myskill", &Skills::mySkillMotion);
```

Step 3: (Optional) Add Bluetooth Command

In `Constants.h`, add to `BtCommand` namespace:

```
static constexpr char MY_SKILL = 'G'; // Pick unused character
```

In `Bluetooth.cpp`, add to `parseCommand()` :

```
case BtCommand::MY_SKILL:
    outState = RobotState::WAVE; // Or add new state
    return true;
```

Future Expansion

The architecture is designed so these features can be added **without major rewrites**:

Feature	Integration Point	Difficulty
MPU6050 / ICM20948 IMU	Add balance correction in <code>Robot::update()</code>	Medium
OLED Display	New FreeRTOS task showing state/battery	Easy
Buzzer	Add to <code>Robot::handleStateTransition()</code>	Easy
Battery Monitoring	ADC reading in safety check loop	Easy
OTA Updates	Enable WiFi + ArduinoOTA in setup	Medium
Wi-Fi Control	Add WebSocket server alongside BT	Medium
ESP-NOW	Multi-robot communication	Medium
Camera (ESP32-CAM)	Replace ESP32 DevKit with CAM module	Hard
Inverse Kinematics	Replace angle-based gait with Cartesian IK	Medium
ROS2 Integration	micro-ROS serial bridge	Hard
Computer Vision	External camera + RPi companion	Hard
AI Navigation	Reinforcement learning on companion	Research

Project Files

```
OpenCatESP32/
|
└─ OpenCatESP32.ino           Main sketch – FreeRTOS tasks, serial commands
```

└─ Config.h	All configurable parameters (namespaced)
└─ Constants.h	Enums, named constants, mapping helpers
└─ Utils.h / .cpp	Math helpers, conversions, debug macros
└─ ServoDriver.h / .cpp	PCA9685 abstraction layer
└─ Calibration.h / .cpp	NVS-persistent servo offset system
└─ Interpolation.h / .cpp	6 interpolation curve types
└─ MotionEngine.h / .cpp	Keyframe-based motion playback
└─ MotionPlayer.h / .cpp	Motion orchestrator with transitions
└─ Skills.h / .cpp	Skill name → Motion registry
└─ Gaits.h / .cpp	Procedural phase-based gait generator
└─ Bluetooth.h / .cpp	BT Classic command reception & parsing
└─ Robot.h / .cpp	Central state machine & coordinator
└─ skills/	Skill definition files
└─ stand.h	Standing pose (1 keyframe)
└─ sit.h	Sitting pose (2 keyframes)
└─ lay.h	Laying down (2 keyframes)
└─ sleep.h	Sleep/compact (2 keyframes)
└─ walk.h	Walk gait (8 keyframes, loop)
└─ back.h	Backward walk (8 keyframes, loop)
└─ left.h	Turn left (4 keyframes, loop)
└─ right.h	Turn right (4 keyframes, loop)
└─ trot.h	Trot gait (4 keyframes, loop)
└─ crawl.h	Slow crawl (8 keyframes, loop)
└─ wave.h	Wave gesture (4 keyframes)
└─ shake.h	Handshake (6 keyframes)
└─ stretch.h	Cat stretch (4 keyframes)
└─ README.md	This file

Total: 37 files · ~120 KB source code

License

Open source. Free to use, modify, and distribute.

Built for makers and robotics enthusiasts

ESP32 · PCA9685 · SG90 · Arduino · FreeRTOS · Bluetooth Classic